

GUIA DE USO

Prospector de Sites

Achou → Refez → Publicou → Ofertou

Este guia explica como instalar o plugin e como usar cada comando na ordem certa — do primeiro *setup* até o e-mail de proposta pronto na caixa de rascunhos, passando pela prospecção no Google Maps, o redesign premium das páginas e a publicação automática no **GitHub + Vercel**.

Ciclo completo: */setup* → */prospectar* → */redesenhar* → */editor* (opcional) → */publicar* → */proposta*

Antes de começar

O plugin roda dentro do Claude Code ou do Claude Cowork e depende de algumas contas e conexões já ativas. Confira a lista antes de instalar.

Claude in Chrome	Extensão conectada — usada para navegar no Google Maps durante a prospecção e como método alternativo de publicação.
Conector do Gmail	Necessário para criar os rascunhos (ou enviar) as propostas por e-mail.
Conector do Google Drive	Necessário para salvar a planilha de leads no Google Sheets.
Pasta conectada no Cowork	Onde ficam o arquivo de configuração, os leads e os sites gerados — escolha uma pasta dedicada (ex.: "Clientes").
Conta no GitHub	Cada cliente vira um repositório próprio. Gere um Personal Access Token com escopo "repo".
Conta na Vercel	Recomendado criar com "Continue with GitHub" para já vincular as contas. Gere um token de API e instale a integração Vercel-GitHub uma única vez.

Instalação

No **Claude Code**, rode estes dois comandos no terminal (sessão interativa):

```
/plugin marketplace add ArrecheNeto/PROSPECTOR-DE-SITES  
/plugin install prospector-de-sites@arrecheneto-plugins
```

No **Claude Cowork** (desktop): *Configurações* → *Plugins* → *Adicionar marketplace* → cole a URL do repositório → instale o **prospector-de-sites**.

Segurança

Nenhuma senha ou token é digitado no chat. Você preenche os campos "token" diretamente no arquivo `prospector-config.json`, que fica salvo apenas na sua pasta conectada — nunca é exibido, impresso ou enviado pelo Claude.

O fluxo ideal, em uma página

O ciclo foi pensado para rodar em lotes: você prospecta um grupo de leads, redesenha os melhores, publica e dispara as propostas — depois volta para prospectar o próximo lote, excluindo automaticamente quem já foi avaliado.

1 /setup Configura tudo uma vez	2 /prospectar Encontra os leads no Google Maps	3 /redesenhar Recria as páginas em lote (5+)
4 /editor Ajuste manual opcional	5 /publicar Sobe no GitHub + Vercel	6 /proposta Envia o e-mail de oferta

■ Depois da **/proposta**, o ciclo recomeça em **/prospectar** para o próximo lote de leads — quem já foi avaliado é excluído automaticamente da nova busca.

Onde ficam os dados

prospector-config.json	Preferências e tokens (GitHub/Vercel) — só na sua pasta conectada.
leads.md	Pipeline local dos leads e status (novo → redesenhado → publicado → proposta enviada).
Planilha no Google Sheets	Cópia dos leads compartilhável, gerada/atualizada pelo /prospectar.
sites/[slug]/	Página final + editor visual de cada cliente.
comparar.html	Comparador antes/depois de todos os clientes do lote.
Repositório GitHub por cliente	"[prefixo]-[slug]", publicado como projeto próprio na Vercel.

Os comandos, em detalhe

Cada comando pode ser rodado sozinho a qualquer momento, mas o resultado é melhor seguindo a ordem do fluxo ideal descrito acima.

1**/setup**

Configura o plugin (roda uma vez)

Quando roda: Na primeira utilização, ou quando quiser atualizar preferências/credenciais.

O que faz:

- Confirma (ou pede para conectar) a pasta de trabalho do Cowork.
- Coleta a assinatura da proposta: nome, apresentação e WhatsApp.
- Define nichos padrão de prospecção (sugestão: nutricionistas, psicólogos, advogados, psiquiatras) e a cidade padrão.
- Define quantos leads qualificados buscar por rodada (padrão: 10) e o modo de envio da proposta (rascunho ou envio direto).
- Coleta usuário/organização do GitHub, visibilidade e prefixo dos repositórios, e os tokens do GitHub e da Vercel (preenchidos por você direto no arquivo, nunca no chat).
- Testa a conexão publicando um repositório de teste e confirma a URL pública na Vercel.

Entradas	Saídas
Respostas ao formulário inicial + tokens preenchidos manualmente no config.	prospector-config.json salvo na pasta conectada, com a integração GitHub + Vercel validada.

Dica

Só precisa rodar de novo se quiser trocar nicho padrão, assinatura ou credenciais.

2

`/prospectar [nicho] [cidade]`

Busca leads qualificados no Google Maps

Quando roda: No início de cada novo lote de prospecção.

O que faz:

- Abre o Google Maps via Claude in Chrome e busca "[nicho] em [cidade]".
- Avalia até 25 estabelecimentos (ou até bater a meta de leads) usando o critério ouro: nota ≥ 4.7 , ≥ 40 avaliações, site ativo porém ruim, e e-mail público.
- Descarta automaticamente quem não tem site, tem site bom, ou não tem e-mail — sempre registrando o motivo.
- Exclui da busca quem já foi avaliado em rodadas anteriores (lido de leads.md).
- Para cada candidato qualificado, abre o site em nova aba e anota o motivo objetivo pelo qual ele é ruim.

Entradas	Saídas
Nicho e cidade (argumentos ou os padrões do config).	Planilha nova/atualizada no Google Sheets (link retornado) + leads.md local com ranking, e-mail, telefone e motivo.

Dica

O usuário sempre pode trocar nicho e cidade na hora — nunca fica travado nos padrões do /setup.

3

/redesenhar [leads]

Recria as páginas com estética premium (lote de 5+)

Quando roda: Depois de ter leads qualificados com status "novo" em leads.md.

O que faz:

- Seleciona os leads mais bem ranqueados com status "novo" (mínimo 5 por lote).
- Abre o site original no navegador e extrai TODO o conteúdo real: textos, serviços, contato, redes sociais, paleta de cores, logo e fotos originais (nada é inventado).
- Aplica a estética premium mantendo o conteúdo real do cliente — é uma nova versão da página, não uma página genérica.
- Gera, para cada cliente: a página final (sites/[slug]/[slug].html) e o editor visual (sites/[slug]/[slug]-editor.html).
- Atualiza o comparador antes/depois (comparar.html) com o lote inteiro, sem perder clientes de lotes anteriores.
- Revisa cada página em busca de texto placeholder, links quebrados ou CTAs que não apontem para o contato real.

Entradas	Saídas
Leads com status "novo" em leads.md (ou uma lista específica de URLs/nomes).	sites/[slug]/[slug].html, sites/[slug]/[slug]-editor.html e comparar.html atualizados; status do lead vira "redesenhado".

Dica

É a entrega mais visual do processo — use comparar.html como peça de venda na proposta.

4

`/editor [cliente]`**Ajuste manual opcional, direto no navegador**

Quando roda: Quando quiser trocar algum texto ou imagem antes (ou depois) de publicar.

O que faz:

- Abre (ou regenera) a versão editável da página de um cliente específico.
- Permite clicar em qualquer texto da página para editá-lo diretamente.
- Permite clicar em qualquer imagem para trocá-la por um arquivo do computador (fica embutida na página).
- Um botão "Exportar página" baixa o HTML final limpo, sem a camada de edição.

Entradas	Saídas
Nome do cliente (ou escolha na lista de sites/existentes).	sites/[slug]/[slug]-editor.html gerado/atualizado; ao exportar, substitui sites/[slug]/[slug].html.

Dica

Se a página já estava publicada, rode `/publicar` de novo depois de editar — a Vercel republica sozinha a cada push.

5

`/publicar [cliente|todos]`

Publica no GitHub + Vercel

Quando roda: Depois que a página está redesenhada (e, se quiser, ajustada no `/editor`).

O que faz:

- Confere se os tokens do GitHub e da Vercel estão preenchidos no config — se não, pausa e orienta a preenchê-los.
- Cria um repositório GitHub por cliente: "[prefixoRepo]-[slug]" (visibilidade definida no `/setup`).
- Envia `sites/[slug]/[slug].html` como `index.html` na raiz do repositório (via API do GitHub, com fallback pelo navegador).
- Cria (ou confirma) o projeto Vercel vinculado ao repositório — isso já dispara o primeiro deploy automaticamente.
- Verifica a URL pública (`https://[prefixoRepo]-[slug].vercel.app`) e confirma que a página carrega.

Entradas	Saídas
Nome do cliente ou "todos"; ou a lista de leads com status "redesenhado".	Repositório GitHub + projeto Vercel por cliente; URL pública; status do lead vira "publicado" em <code>leads.md</code> .

Dica

Depois do primeiro deploy, qualquer novo push no mesmo repositório republica sozinho — não precisa recriar nada.

6

`/proposta [cliente|todos]`**Escreve e envia o e-mail de proposta**

Quando roda: Depois que a página do cliente já está publicada com URL pública.

O que faz:

- Seleciona os leads com status "publicado" que ainda não receberam proposta.
- Escreve o e-mail com elogio real baseado nas avaliações do Google, o defeito específico apontado na prospecção, o link do site antigo e o link da nova página — sem mencionar preço.
- Tira um screenshot da nova página e salva em `sites/[slug]/preview.png`, para anexar e aumentar a taxa de abertura.
- Cria o rascunho no Gmail (padrão) ou envia direto, conforme o modo definido no `/setup`.
- Para leads sem e-mail confirmado, sugere abordagem manual pelo WhatsApp com texto adaptado.

Entradas	Saídas
Nome do cliente ou "todos"; dados do lead (avaliações, defeito do site antigo, URLs) e a assinatura do config.	Rascunho (ou envio) criado no Gmail; status do lead vira "proposta enviada" + data em <code>leads.md</code> .

Dica

O fechamento (preço, reunião) acontece na resposta do cliente — combine um follow-up educado após 3-4 dias úteis sem resposta.

Boas práticas para o fluxo ideal

- Rode em lotes de 5-10 leads por vez — é o tamanho pensado para `/redesenhar` e mantém o `comparar.html` gerenciável.
- Nunca pule o `/redesenhar` direto para o `/publicar` sem revisar `comparar.html` — é a peça que mais convence na proposta.
- Use `/editor` para pequenos ajustes finos (erro de digitação, trocar uma foto) em vez de pedir um redesign completo de novo.
- Depois de qualquer edição em uma página já publicada, rode `/publicar` de novo para sincronizar — a Vercel cuida do resto.
- Mantenha o modo de envio da proposta como "rascunho" até se sentir confiante com a qualidade do e-mail gerado.
- Ao final de um lote, volte para `/prospectar` — leads já avaliados (qualificados ou descartados) nunca se repetem.